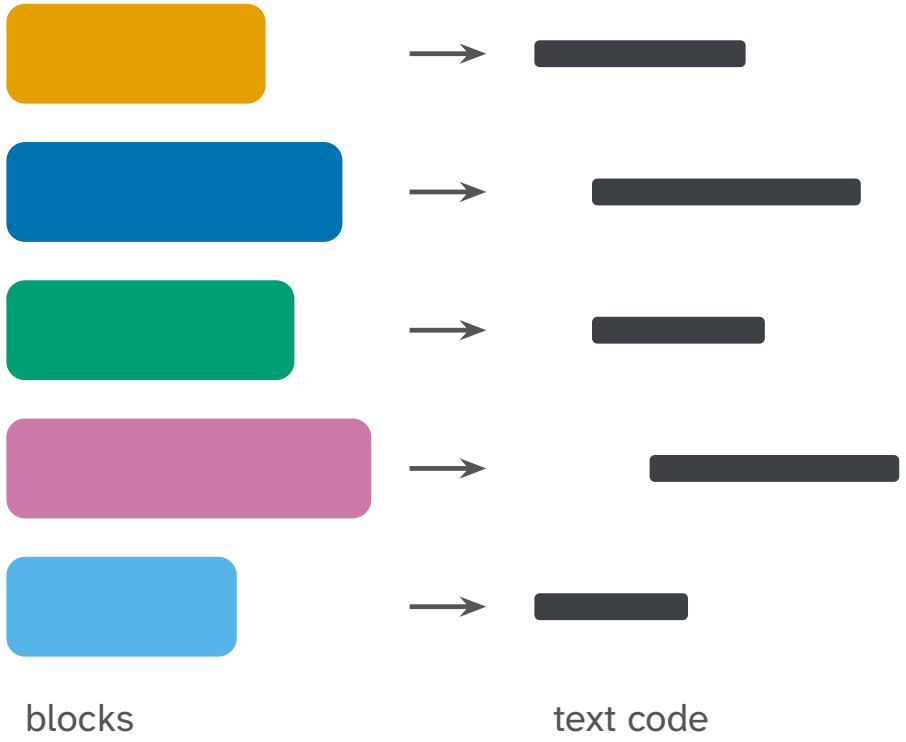
FAMILY INFORMATION EVENING

# Why Coding?

## For families of Grade 5 learners

What the research says, how our course works, and how you can help at home.

[Presenter name] | Kids Coding course | starts January 2027



TONIGHT

# Tonight in 25 minutes

## 1

### Why now

Computing, AI and the skills ahead

## 2

### The evidence

What studies show, and what they don’t

## 3

### Good teaching

What works, according to research

## 4

### Our course

Three levels, one session a week

## 5

### Your role

Easy ways to help at home

**Then**

Questions, common concerns and next steps. Ask anything, any time.

WHY NOW

# Computing is changing the skills our kids need

Employers expect these to be the **fastest-growing skills to 2030**:

1. **AI and big data**
2. **Networks and cybersecurity**
3. **Technological literacy**

**Analytical thinking** is the top *core* skill: 7 in 10 employers call it essential.

> **Take-away**
>
> Not every child should become a programmer, but every child will live with this technology.

**Source:** World Economic Forum (2025), a survey of over 1,000 employers.

WHY NOW

# Schools are still catching up

**60%**

of US public high schools offered a foundational computer science class in 2024–25. Access has **stopped growing**.

**6.1%**

of high school students were enrolled in one. Most students never take a class.

> **Take-away**
>
> A small, steady start outside school fills a gap that many schools cannot yet fill.

**Source:** Code.org et al. (2025); national data, 2024–25 school year.

WHY NOW

# With AI, understanding code matters more

### 69 learners aged 10–17

- With an AI helper, they wrote more working code.
- They could still edit code by hand.
- A week later: no clear difference.

### OECD report, 2026

- General chatbots can lift scores without lasting learning.
- Tools designed for learning work better.

**Open question** How AI will change learning to code is not yet known.

**Take-away**

Someone still has to understand, check and fix what the computer produces.

**Source:** Kazemitabaar et al. (2023); OECD (2026).

WHAT THE RESEARCH SHOWS

# What coding builds, and how sure we are

| | | |
|---|---|---|
| **Coding and computational thinking** | learns to program | Strong evidence |
| **Problem solving and planning** | on coding-like tasks | Strong evidence |
| **Math, creativity, reasoning** | smaller in strict studies | Mixed evidence |
| **Reading and school grades** | little effect | Mixed evidence |
| **General persistence and teamwork** | rarely measured | Open question |

**What "computational thinking" means**

Breaking a problem into steps, spotting patterns, and testing ideas until they work.

**Source:** Scherer et al. (2019, 2020); Montuori et al. (2024); Wing (2006).

WHAT THE RESEARCH SHOWS

# The biggest review: 105 studies

- Children reliably **learn to program** when taught.
- Some gains spread: **creative thinking, math, thinking about thinking**.
- Smaller gains for spatial skills and reasoning.
- Least benefit: **reading and school grades**.
- Stronger study designs found **smaller effects**.

> **Take-away**
>
> Expect better coders and problem solvers, not automatically higher grades.

**Source:** Scherer et al. (2019) (105 studies, 539 effects); Scherer et al. (2020).

WHAT THE RESEARCH SHOWS

# Where coding seems to help most

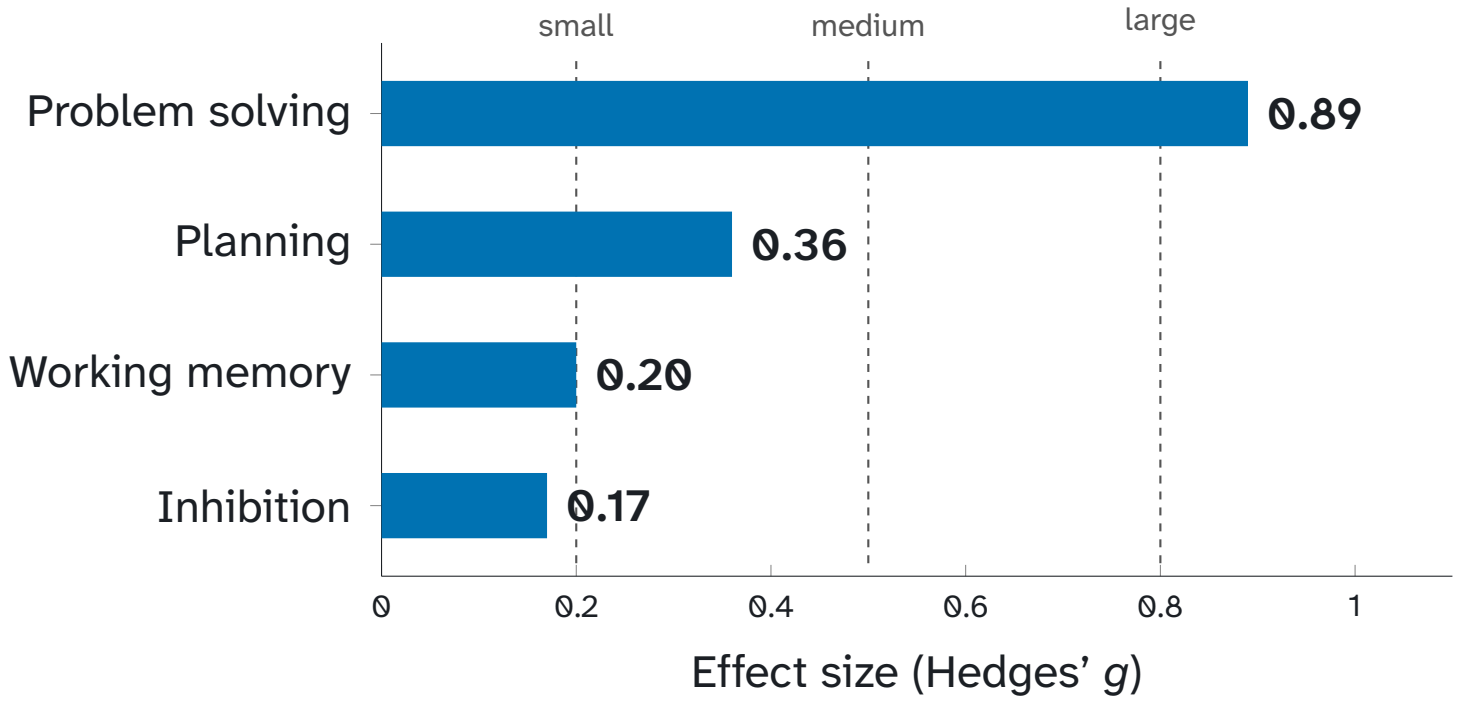



*Figure:* bar chart of effect sizes: problem solving 0.89, planning 0.36, working memory 0.20, inhibition 0.17. Dashed lines mark the usual small, medium and large benchmarks.

**19 experiments** with children; 11 pooled.

Big gain in **problem solving**; small gains in self-control and memory.

◐ **Mixed evidence**

Few studies; effects vary.

**Source:** Montuori et al. (2024); pooled (fixed-effect) estimates from 11 studies.

# A closer look: one fair test

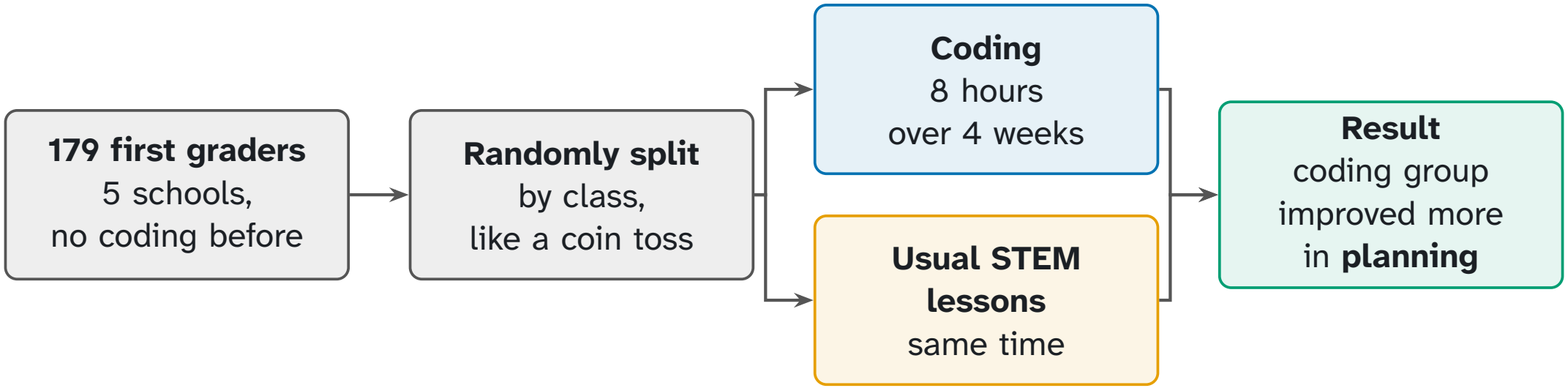



*Figure:* flow diagram of a randomized trial: 179 first graders randomly assigned by class to coding or usual STEM lessons; the coding group improved more in planning.

**Limits:** young children, a short program, one country (Italy). Gains in self-control were smaller.

**Source:** Arfé et al. (2020), a cluster-randomized controlled trial.

WHAT THE RESEARCH SHOWS

# Early experiences shape interest

### Programming a robot, age 6

Afterwards, girls were as interested and confident as boys.

Stereotypes did not change.

### Plans at age 13–14

Expecting a science career predicted a science degree (2–3×).

A link, not proof of cause.

Girls report less encouragement in computing from parents and teachers.

**Take-away**

Early, positive experiences keep doors open. They do not decide a future.

**Source:** Master et al. (2017); Tai et al. (2006); Google and Gallup (2016).

# What we don’t know (yet)

- Gains in other school subjects are **mixed and often small**.
- Stronger studies find **smaller benefits**.
- Skills rarely carry far: the same is true of **chess and music**.
- Kids who choose coding may differ: **correlation is not cause**.
- Results depend heavily on **how it is taught**.
- We make **no promises** about jobs, pay or college.

> **Our promise**
>
> Capable, confident beginner programmers who practice problem solving every week.

**Source:** Pea and Kurland (1984) (an early critique); Sala and Gobet (2017); Scherer et al. (2019).

WHAT GOOD TEACHING LOOKS LIKE

# Six principles the evidence supports



**Real projects**

Games and art they love

Resnick et al., 2009



**Low floor, high ceiling**

Easy start, room to grow

Resnick et al., 2009



**Code in pairs**

Take turns at the keys

Umapathy & Ritzhaupt, 2017



**Teach debugging**

A step-by-step bug hunt

Michaeli & Romeike, 2019



**Off-screen, read first**

Predict, then run

Sentance et al., 2019



**Spiral, with feedback**

Ideas return, deeper

Bruner, 1960

# Create, don’t just consume

- Kids learn best making things **they care about**.
- **Low floor:** a first program in minutes.
- **High ceiling:** real complexity later.
- **Wide walls:** games, art, stories, data.

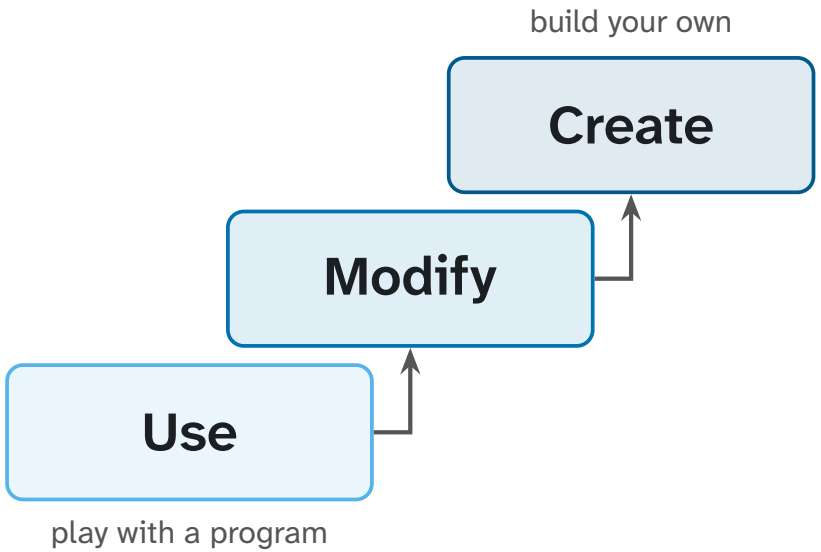



*Figure:* a staircase from Use to Modify to Create.

**Take-away**

Every level ends with a project the child designs, builds and presents.

**Source:** Papert (1980); Resnick et al. (2009); Lee et al. (2011).

# Learn together, fix bugs on purpose

### Pairs

Better assignments, exams and persistence. Beginners gain most.

### Debugging

A taught, step-by-step process improved skill **and** confidence.

### Feedback

Helps most when it says **how to improve**.

**Strong evidence** Pairs and feedback.

**Mixed evidence** Debugging: promising, but few studies.

**Source:** Umapathy and Ritzhaupt (2017); Denner et al. (2014); Michaeli and Romeike (2019); Wisniewski et al. (2020).

# Off-screen first, read before writing, return often

### Unplugged

Starting off-screen helped young pupils' thinking and motivation.

### Predict, then run

Students aged 11–14 who read code first beat a comparison group.

### Spiral

Big ideas come back each level, deeper each time.

**Loops, for example:** Scratch *repeat* → Python `for` → game loops → recursion.

**Source:** del Olmo-Muñoz et al. (2020); Sentance et al. (2019); Bruner (1960).

OUR COURSE

# How our course puts this into practice

| Principle | In our course |
|---|---|
| **Real projects** | A final project every level; 10+ free-build times |
| **Low floor, high ceiling** | Scratch, then Python; easier and harder paths |
| **Code in pairs** | Driver and navigator, swapped every session |
| **Teach debugging** | A “bug hunt” every week, with a strategy |
| **Off-screen, read first** | 6+ unplugged activities; predict-then-run |
| **Spiral, with feedback** | Ideas return each level; exit check each lesson |

**Source:** Course design (docs/DESIGN.md, docs/SYLLABUS.md); per-level minimums are checked automatically.

OUR COURSE

# Why blocks first, then Python

- Blocks remove typing errors: kids focus on ideas.
- Teens learned more with blocks at first.
- After switching to text, the gap disappeared.
- So: 6 weeks of Scratch, then Python.

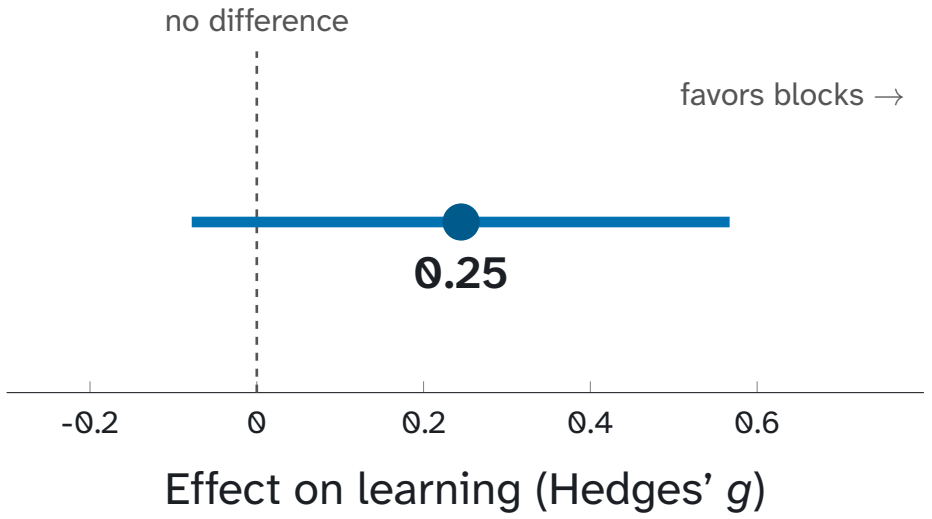


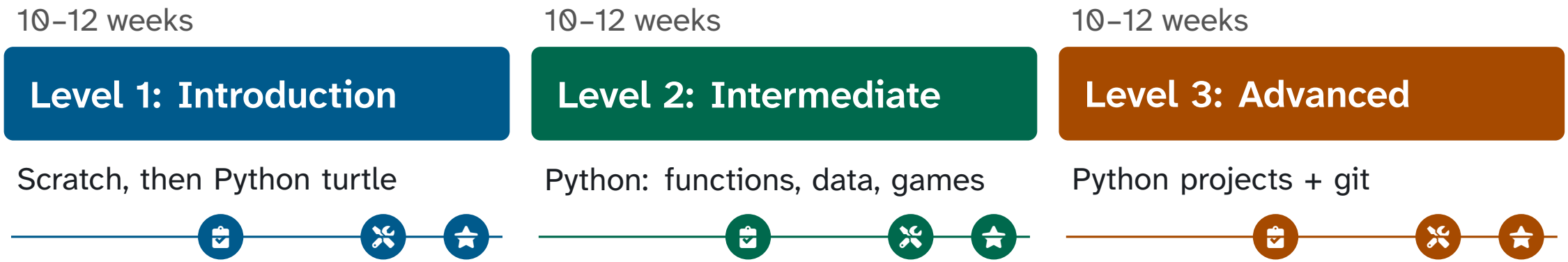

*Figure:* estimate 0.25 with a range from −0.08 to 0.57 that crosses zero: a small edge for blocks that could be chance.

**Mixed evidence** Review of 13 studies: a small edge for blocks, could be chance.

**Source:** Weintrop and Wilensky (2017, 2019); Xu et al. (2019).

OUR COURSE

# The roadmap: three levels of 10–12 weeks




One session a week. 10-week plan: two pairs of weeks are combined. Holiday weeks don't count.

*Figure:* timeline of three equal levels of 10–12 weeks each: Level 1 Introduction, Level 2 Intermediate, Level 3 Advanced, each with a checkpoint in week 6, a final project in weeks 10–11 and a showcase in week 12.

**Take-away**

No experience needed: everyone starts at Level 1, week 1.

OUR COURSE

# Level 1: Introduction

**Scratch blocks, then Python turtle · weeks 1–6 in blocks, 7–12 in Python**

**Your child will learn to…**

- give exact step-by-step instructions
- repeat with loops; react with “if”
- keep score with variables
- find and fix bugs with a strategy
- write first Python programs

**Final project**

**My Interactive Art Show**

Asks the viewer questions, then draws a scene from the answers.

**Along the way:** catch games, star patterns, a magic dice.

**Progress checks:** welcome survey (ungraded), week 6, week 12, project rubric.

**Source:** Level 1 syllabus; aligned with CSTA standards for grades 3–5 (Computer Science Teachers Association, 2017).

OUR COURSE

# Level 2: Intermediate

**Python in Thonny, a free beginner editor**

**Your child will learn to…**

- organize code into reusable functions
- store data in lists and dictionaries
- save and load files, like high scores
- write tests that check code automatically
- build keyboard-controlled games

**★ Final project**

**Arcade Game or Art Studio**

A game with saved high scores, or an art studio, plus the child's own tests.

**Along the way:** secret-code machines, pixel art, guess-the-number.

**Progress checks:** placement quiz, week 6, week 12, project rubric.

**Source:** Level 2 syllabus; aligned with CSTA standards for grades 6–8 (Computer Science Teachers Association, 2017).

OUR COURSE

# Level 3: Advanced

**Python projects + git · git is a “save-game” system for code**

**Your child will learn to…**

- design programs with classes and objects
- compare ways to search and sort
- draw fractals with recursion
- clean and chart a small data set
- explain how simple AI learns, and fails

**Final project**

**Build Something Big**

A game, art app or AI app with classes, an algorithm and tests.

**Along the way:** virtual pets, fractal trees, a tiny AI.

**Progress checks:** placement quiz, week 6, week 12, project rubric.

**Source:** Level 3 syllabus; AI topics follow the AI4K12 Five Big Ideas (Touretzky et al., 2019).

OUR COURSE

# What a 60-minute session looks like




*Figure:* five-part session: warm-up 8 minutes, new idea 10, demo 10, building in pairs 24, show and tell 8.

**Homework**

15–20 min a week, optional

**Equipment**

A Mac with free apps; no accounts

**Group**

Up to 4 kids in 2 pairs

OUR COURSE

# What success looks like

| By the end of | most children can… |
|---|---|
| Level 1 | build a small game and a drawing program, and explain them |
| Level 2 | organize a game into functions, save scores, test code |
| Level 3 | plan a multi-part project, track it in git, explain choices |

**How we check:** checkpoint in week 6, exit check in week 12, project rubric.

**Rubric:** Just starting · Getting there · Got it · Expert

> **No grades**
> Scores guide support. Creativity, working code and explaining it count equally.

YOUR ROLE

# How you can help, no coding needed

- **Ask for a demo:** “Show me what you made.”
- **Ask about the bug,** not the score: “What went wrong? How did you find it?”
- **Praise strategies and effort,** not “being a natural.”
- **Protect 15 minutes** a week for homework, at a regular time.
- **Encourage everyone equally:** girls hear less encouragement than boys.
- **Come to the showcase** at the end of each level.

**Source:** Google and Gallup (2016); Master et al. (2017); Wisniewski et al. (2020).

# Safe, private and inclusive by design

- **No accounts, no logins, no email addresses.**
- **Nicknames only;** programs never ask for personal information.
- **Nothing is uploaded;** projects stay on the computers in the room.
- **Screen breaks every session,** plus unplugged activities.
- **Color-blind-safe materials** and a dyslexia-friendly edition.

**Why no accounts?**

US law requires parental consent before online services collect data from children under 13. We collect nothing.

**Source:** Federal Trade Commission (2025); AAP Council on Communications and Media (2016); Okabe and Ito (2008).

QUESTIONS

# Common questions

### Isn’t 10 too young?

No. Scratch was designed for this age.

### More screen time?

One hour a week, with breaks and games.

### Won’t AI make this pointless?

Someone has to understand and check AI’s work.

### I can’t code. Can I help?

Yes: ask questions and praise effort.

QUESTIONS

# Next steps

1. **Take the handout** home and talk it over with your child.
2. **Let us know** if your child will join Level 1 (starts January 2027).
3. **Check the computer:** we help install Scratch and Thonny.

> **Questions?**
> [Presenter name] | [email or phone]

A

# Appendix

Evidence summary, notes on sources, credits and references

APPENDIX

# Evidence at a glance

| Claim | Strength | Key sources |
|---|---|---|
| Children learn to program when taught | Strong evidence | Scherer et al. (2020) |
| Gains in problem solving and planning | Strong evidence | Montuori et al. (2024); Arfé et al. (2020) |
| Gains in math, creativity, reasoning | Mixed evidence | Scherer et al. (2019) |
| Gains in reading and school grades | Mixed evidence | Scherer et al. (2019) |
| Pair programming improves learning | Strong evidence | Umapathy and Ritzhaupt (2017); Denner et al. (2014) |
| Blocks before text help learning | Mixed evidence | Xu et al. (2019); Weintrop and Wilensky (2019) |
| Explicit debugging teaching helps | Mixed evidence | Michaeli and Romeike (2019) |
| Coding builds general persistence | Open question | no strong studies found |
| Long-term effect of AI on learning to code | Open question | Kazemitabaar et al. (2023); OECD (2026) |

APPENDIX

# Notes on dates and limits

- **Older sources:** Pea and Kurland (1984), Papert (1980) and Bruner (1960) are cited for ideas, not current data.
- **Tai et al. (2006)** uses data from 1988–2000; it shows an association only.
- **Most pair-programming studies** were with university students.
- **Hattie & Timperley (2007)** reported 0.79 for feedback; a larger re-analysis found about 0.48 (Wisniewski et al., 2020).
- **CSTA** published a draft revision of its standards in 2025; the 2017 version is cited.
- **Survey data** (WEF, Code.org, Google–Gallup) describe expectations and access, not outcomes.

APPENDIX

# Credits and licenses

- **All diagrams and charts:** original, drawn in TikZ/pgfplots for this deck.
- **Chart data:** published values only, from the cited papers.
- **Fonts:** Atkinson Hyperlegible Next and Mono (Braille Institute), SIL Open Font License 1.1.
- **Icons:** Font Awesome 5 Free via the fontawesome5 package; icons CC BY 4.0, font SIL OFL 1.1.
- **Colors:** Okabe–Ito color-blind-safe palette (Okabe & Ito, 2008).
- **Trademarks:** Scratch is a project of the Scratch Foundation; Python is a trademark of the PSF.

APPENDIX

# References I


AAP Council on Communications and Media. (2016). Media use in school-aged children and adolescents. *Pediatrics*, *138*(5), e20162592. https://doi.org/10.1542/peds.2016-2592

Arfé, B., Vardanega, T., & Ronconi, L. (2020). The effects of coding on children's planning and inhibition skills. *Computers & Education*, *148*, 103807. https://doi.org/10.1016/j.compedu.2020.103807

Bruner, J. S. (1960). *The process of education*. Harvard University Press.

Code.org, CSTA, & ECEP Alliance. (2025, December). *2025 state of AI & computer science education*. Code.org Advocacy Coalition. https://advocacy.code.org/stateofcs/

Computer Science Teachers Association. (2017). *CSTA K–12 computer science standards, revised 2017*. CSTA. https://csteachers.org/k12standards/

del Olmo-Muñoz, J., Cózar-Gutiérrez, R., & González-Calero, J. A. (2020). Computational thinking through unplugged activities in early years of primary education. *Computers & Education*, *150*, 103832. https://doi.org/10.1016/j.compedu.2020.103832

Denner, J., Werner, L., Campe, S., & Ortiz, E. (2014). Pair programming: Under what conditions is it advantageous for middle school students? *Journal of Research on Technology in Education*, *46*(3), 277–296. https://doi.org/10.1080/15391523.2014.888272

Federal Trade Commission. (2025, April 22). *Children's online privacy protection rule (16 cfr part 312), as amended* [Amendments effective June 23, 2025]. https://www.ecfr.gov/current/title-16/chapter-I/subchapter-C/part-312

Google & Gallup. (2016). *Diversity gaps in computer science: Exploring the underrepresentation of girls, Blacks and Hispanics*. Google Inc. https://services.google.com/fh/files/misc/diversity-gaps-in-computer-science-report.pdf

APPENDIX

# References II


Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3544548.3580919

Lee, I., Martin, F., Denner, J., Coulter, B., Allan, W., Erickson, J., Malyn-Smith, J., & Werner, L. (2011). Computational thinking for youth in practice. *ACM Inroads*, *2*(1), 32–37. https://doi.org/10.1145/1929887.1929902

Master, A., Cheryan, S., Moscatelli, A., & Meltzoff, A. N. (2017). Programming experience promotes higher STEM motivation among first-grade girls. *Journal of Experimental Child Psychology*, *160*, 92–106. https://doi.org/10.1016/j.jecp.2017.03.013

Michaeli, T., & Romeike, R. (2019). Improving debugging skills in the classroom: The effects of teaching a systematic debugging process. *Proceedings of the 14th Workshop in Primary and Secondary Computing Education (WiPSCE '19)*, 57–63. https://doi.org/10.1145/3361721.3361724

Montuori, C., Gambarota, F., Altoè, G., & Arfé, B. (2024). The cognitive effects of computational thinking: A systematic review and meta-analytic study. *Computers & Education*, *210*, 104961. https://doi.org/10.1016/j.compedu.2023.104961

OECD. (2026). *OECD digital education outlook 2026: Exploring effective uses of generative AI in education*. OECD Publishing. Paris. https://doi.org/10.1787/062a7394-en

Okabe, M., & Ito, K. (2008). *Color universal design (CUD): How to make figures and presentations that are friendly to colorblind people*. https://jfly.uni-koeln.de/color/

Papert, S. (1980). *Mindstorms: Children, computers, and powerful ideas*. Basic Books.

APPENDIX

# References III


Pea, R. D., & Kurland, D. M. (1984). On the cognitive effects of learning computer programming. *New Ideas in Psychology*, *2*(2), 137–168. https://doi.org/10.1016/0732-118X(84)90018-7

Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., Millner, A., Rosenbaum, E., Silver, J., Silverman, B., & Kafai, Y. (2009). Scratch: Programming for all. *Communications of the ACM*, *52*(11), 60–67. https://doi.org/10.1145/1592761.1592779

Sala, G., & Gobet, F. (2017). Does far transfer exist? Negative evidence from chess, music, and working memory training. *Current Directions in Psychological Science*, *26*(6), 515–520. https://doi.org/10.1177/0963721417712760

Scherer, R., Siddiq, F., & Sánchez Viveros, B. (2019). The cognitive benefits of learning computer programming: A meta-analysis of transfer effects. *Journal of Educational Psychology*, *111*(5), 764–792. https://doi.org/10.1037/edu0000314

Scherer, R., Siddiq, F., & Sánchez Viveros, B. (2020). A meta-analysis of teaching and learning computer programming: Effective instructional approaches and conditions. *Computers in Human Behavior*, *109*, 106349. https://doi.org/10.1016/j.chb.2020.106349

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education*, *29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781

Tai, R. H., Liu, C. Q., Maltese, A. V., & Fan, X. (2006). Planning early for careers in science. *Science*, *312*(5777), 1143–1144. https://doi.org/10.1126/science.1128690

Touretzky, D., Gardner-McCune, C., Martin, F., & Seehorn, D. (2019). Envisioning AI for K–12: What should every child know about AI? *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*, 9795–9799. https://doi.org/10.1609/aaai.v33i01.33019795

APPENDIX

# References IV


Umapathy, K., & Ritzhaupt, A. D. (2017). A meta-analysis of pair-programming in computer programming courses: Implications for educational practice. *ACM Transactions on Computing Education*, *17*(4), 16:1–16:13. https://doi.org/10.1145/2996201

Weintrop, D., & Wilensky, U. (2017). Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education*, *18*(1), 3:1–3:25. https://doi.org/10.1145/3089799

Weintrop, D., & Wilensky, U. (2019). Transitioning from introductory block-based and text-based environments to professional programming languages in high school computer science classrooms. *Computers & Education*, *142*, 103646. https://doi.org/10.1016/j.compedu.2019.103646

Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, *49*(3), 33–35. https://doi.org/10.1145/1118178.1118215

Wisniewski, B., Zierer, K., & Hattie, J. (2020). The power of feedback revisited: A meta-analysis of educational feedback research. *Frontiers in Psychology*, *10*, 3087. https://doi.org/10.3389/fpsyg.2019.03087

World Economic Forum. (2025, January). *The future of jobs report 2025*. World Economic Forum. Geneva. https://www.weforum.org/publications/the-future-of-jobs-report-2025/

Xu, Z., Ritzhaupt, A. D., Tian, F., & Umapathy, K. (2019). Block-based versus text-based programming environments on novice student learning outcomes: A meta-analysis study. *Computer Science Education*, *29*(2–3), 177–204. https://doi.org/10.1080/08993408.2019.1565233